Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

# Лабораторная работа №1 по курсу

# «Операционные системы»

Группа: М8О-208БВ-25

Студент: Иудин Е.В.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 29.09.26

Москва, 2026

# Постановка задачи

**Вариант 8.**

В файле записаны команды вида:«число число число<endline>». Дочерний процесс производит деление первого числа команда, на последующие числа в команде, а результат выводит в стандартный поток вывода. Если происходит деление на 0, то тогда дочерний и родительский процесс завершают свою работу. Проверка деления на 0 должна осуществляться на стороне дочернего процесса. Числа имеют тип int. Количество чисел может быть произвольным.

# Общий метод и алгоритм решения

Использованные системные вызовы:

- pid_t fork(void); – создает дочерний процесс.
- int pipe(int *fd); – создает однонаправленный канал связи.
- ssize_t read(int fd, void *buf, size_t count); – чтение данных из файлового дескриптора.
- ssize_t write(int fd, const void *buf, size_t count); – запись данных в файловый дескриптор.
- void _exit(int status); – немедленное завершение текущего процесса с кодом status.
- int close(int fd); – закрывает открытый файловый дескриптор.
- pid_t wait(int *status); – заставляет родительский процесс приостановиться и ждать, пока не завершится любой из дочерних процессов.
- int dup2(int oldfd, int newfd); – дублирует файловый дескриптор oldfd под новым newfd.
- int open(const char *pathname, int flags, mode_t mode); – создает или открывает файл по пути pathname и возвращает его файловый дескриптор.
- int execl(const char *path, const char *arg0, ..., (char *)NULL); – ОС полностью стирает текущую программу, выполняющуюся в текущем процессе, и запускает новую по pathname.

Структура такова: в parent.c описываем родительский процесс, а в child.c наш дочерний процесс, который подгружается после execl().

Для начала мы считываем имя файла, в котором будут находиться команды (его имя не превышает 255 символов). Далее, пытаемся открыть файл с помощью системного вызова open(). После этого создаем однонаправленный канал pipe для межпроцессного взаимодействия с помощью системного вызова pipe(). Затем вызываем fork() для создания дочернего процесса и на стороне дочернего процесса вызываем execl(), перед этим с помощью dup2() перенаправив стандартный ввод к файлу, а вывод к pipe. Не забываем закрыть файловые дескрипторы. На стороне родительского процесса просто будем читать из pipe и отправлять в стандартный вывод. На стороне дочернего процесса с помощью кольцевого буфера размером 2048 байт читаем содержимое файла по 1024 байта с помощью read(). Парсим числа из буфера и производим деление первого на все последующие. В случае если мы обрабатываем уже не первое число в строке и нам

встречается ноль, то пишем в pipe что произошло деление на ноль и вызываем _exit(1). В случае если строка закончилась без происшествий, просто отправляем результат деления первого числа на все последующие в строке в pipe.

# Код программы

**parent.c**

```
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

void write_string(int fd, char *s) {
    size_t len = 0;
    while (s[len] != '\0') len++;
    write(fd, s, len);
}

int main() {
    int status;
    char filename[256];

    write_string(STDOUT_FILENO, "Введите имя файла (max 255 символов): ");

    size_t b_read = 0;
    char ch;

    while(b_read < sizeof(filename) - 1) {
        int res = read(STDIN_FILENO, &ch, 1);
        if (res <= 0 || ch == '\n' || ch == '\r') break;
        filename[b_read++] = ch;
    }
```

```
filename[b_read] = '\0';

if (b_read == 0) {
    write_string(STDOUT_FILENO, "пустой файл");
    _exit(1);
}

int file_fd = open(filename, O_RDONLY);
if (file_fd < 0) {
    write_string(STDOUT_FILENO, "не открывается файл");
    _exit(1);
}

int pipe_[2];
if (pipe(pipe_) < 0) {
    write_string(STDOUT_FILENO, "не удалось создать pipe");
    close(file_fd);
    _exit(1);
}

pid_t pid = fork();

if (pid < 0) {
    write_string(STDOUT_FILENO, "Не удалось сделать fork");
    close(file_fd);
    close(pipe_[0]);
    close(pipe_[1]);
    _exit(1);
}
```

```
if (pid == 0) {
    dup2(file_fd, STDIN_FILENO);
    dup2(pipe_[1], STDOUT_FILENO);

    close(file_fd);
    close(pipe_[1]);
    close(pipe_[0]);

    execl("./child", "child", NULL);

    write_string(STDOUT_FILENO, "Exec неуспешен");
    _exit(1);
}

else {
    close(file_fd);
    close(pipe_[1]);

    char buffer[512];
    size_t count ;

    while ((count = read(pipe_[0], buffer, sizeof(buffer))) > 0) {
        write(STDOUT_FILENO, buffer, count);
    }

    close(pipe_[0]);
    wait(&status);
}
```

```
    _exit(0);
}
```

**child.c**

```
#include <unistd.h>
#include <stdbool.h>
#include <stdio.h>

#define BUF_SIZE 2048
#define CHUNK_SIZE 1024

int main(void) {
    char buf[BUF_SIZE];
    int head = 0, tail = 0, bytes = 0;
    bool eof = false, last = false, num_ends = false;
    int num = 0, res = 0;
    int sign = 1, cnt = 0;

    while (!last) {
        if (!eof && BUF_SIZE - bytes >= CHUNK_SIZE) {
            int n = BUF_SIZE - tail;
            if (n > CHUNK_SIZE) n = CHUNK_SIZE;
            int r = read(STDIN_FILENO, buf + tail, n);
            if (r <= 0) {
                eof = true;
            } else {
                tail = (tail + r) % BUF_SIZE;
                bytes += r;
            }
        }
```

```
char c;
if (bytes > 0) {
    c = buf[head];
    head = (head + 1) % BUF_SIZE;
    bytes--;
} else if (eof) {
    c = '\n';
    last = true;
} else {
    continue;
}

if (c >= '0' && c <= '9') {
    num = num * 10 + (c - '0');
    num_ends = true;
    continue;
}

if (num_ends) {
    if (cnt > 0 && num == 0) {
        write(STDOUT_FILENO, "Ошибка: деление на ноль\n", 44);
        _exit(1);
    }
    res = cnt > 0 ? res / num * sign : num * sign;
    cnt++;
    num = 0;
    num_ends = false;
    sign = 1;
}
```

```
        if (c == '-') sign = -1;


        if (c == '\n') {
            if (cnt > 0) {
                char out[32];
                int len = snprintf(out, sizeof(out), "%d\n", res);
                write(STDOUT_FILENO, out, len);
            }
            cnt = 0;
            res = 0;
            sign = 1;
        }
    }


    _exit(0);
}
```

# Протокол работы программы

Для работы программы, необходимо предварительно скомпилировать parent.c и child.c, причем child.c с именем child.

Для следующего входного файла:

```
7 1 1
8 2 2 2
9 3 1
8 4 1
15 3 5
15 5
18 6
36 2 1 1 1 2
0 9
9 0
18 2
```

Результатом запуска parent.c будет:

```
4oddy@MacBook-Pro-Egor-3 lab1 % gcc -o child child.c && gcc -o parent parent.c && ./parent
Введите имя файла (max 255 символов): file.txt
7
1
3
2
1
3
3
9
0
Ошибка: деление на ноль
```

# Вывод

В ходе выполнения лабораторной работы я научился работать с базовыми системными вызовами в системе Unix. Было изучено межпроцессное взаимодействие с помощью pipe, а также работа с файловыми дескрипторами. При выполнении задания были незначительные трудности, связанные с тем, что раньше этого делать не приходилось.